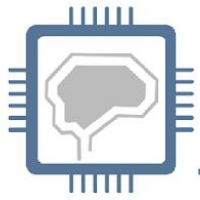


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

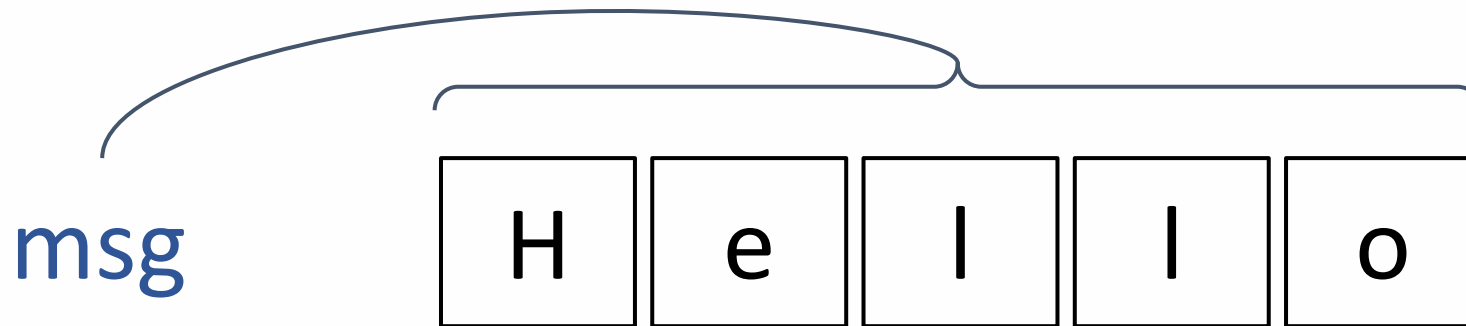
Aula 1: Elementos básicos da linguagem Python

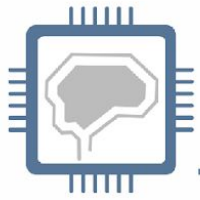
Profa.: Sabrina de A. Silveira



Variáveis

As variáveis são rótulos ou nomes que usamos dentro dos programas para nos referirmos a um espaço de memória RAM onde é armazenado algum dado.





Inteiros

O conceito é semelhante ao usado na Matemática. Com eles podemos realizar diversas operações, como:

Soma

```
>>> a=2
>>> b=3
>>> c = a + b
>>> c
5
```

Subtração

```
>>> d = b - a
>>> d
1
```

Multiplicação

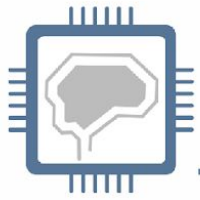
```
>>> e = a * b
>>> e
6
```

Divisão

```
>>> f = b / a
>>> f
1.5
```

Exponenciação

```
>>> g = a ** b
>>> g
8
```



Inteiros

O conceito é semelhante ao usado na Matemática. Com eles podemos realizar diversas operações, como:

Soma

```
>>> a=2
>>> b=3
>>> c = a + b
>>> c
5
```

Subtração

```
>>> d = b - a
>>> d
1
```

Multiplicação

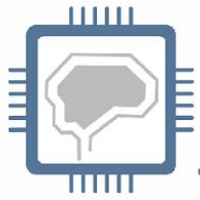
```
>>> e = a * b
>>> e
6
```

Divisão

```
>>> f = b / a
>>> f
1.5
```

Exponenciação

```
>>> g = a ** b
>>> g
8
```



Inteiros

O conceito é semelhante ao usado na Matemática. Com eles podemos realizar diversas operações, como:

Soma

```
>>> a=2
>>> b=3
>>> c = a + b
>>> c
5
```

Subtração

```
>>> d = b - a
>>> d
1
```

Multiplicação

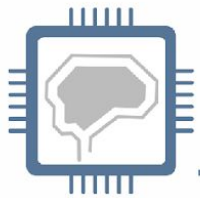
```
>>> e = a * b
>>> e
6
```

Divisão

```
>>> f = b / a
>>> f
1.5
```

Exponenciação

```
>>> g = a ** b
>>> g
8
```



Inteiros

O conceito é semelhante ao usado na Matemática. Com eles podemos realizar diversas operações, como:

Soma

```
>>> a=2
>>> b=3
>>> c = a + b
>>> c
5
```

Subtração

```
>>> d = b - a
>>> d
1
```

Multiplicação

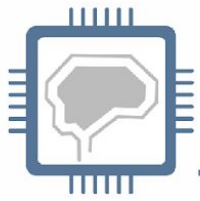
```
>>> e = a * b
>>> e
6
```

Divisão

```
>>> f = b / a
>>> f
1.5
```

Exponenciação

```
>>> g = a ** b
>>> g
8
```



Inteiros

O conceito é semelhante ao usado na Matemática. Com eles podemos realizar diversas operações, como:

Soma

```
>>> a=2
>>> b=3
>>> c = a + b
>>> c
5
```

Subtração

```
>>> d = b - a
>>> d
1
```

Multiplicação

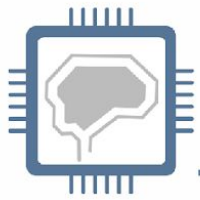
```
>>> e = a * b
>>> e
6
```

Divisão

```
>>> f = b / a
>>> f
1.5
```

Exponenciação

```
>>> g = a ** b
>>> g
8
```

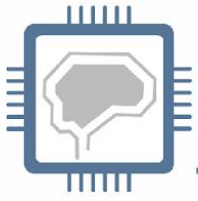
Inteiros

Múltiplas operações podem ser realizadas em uma expressão. Os parênteses podem ser usados para garantir que as operações sejam executadas na ordem desejada.

```
1 → >>> a = 2
>>> b = 3
>>> c = 4
>>> r1 = a + b * c
>>> r1
14
```

```
2 → >>> r2 = (a + b) * c
>>> r2
20
```

```
3 → >>> r3 = a + b * c / 2
>>> r3
8.0
```

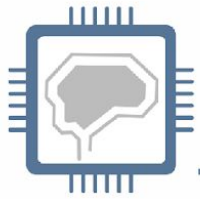
Ponto flutuante

Os números de ponto flutuante ou ponto decimal são chamados de float no contexto da linguagem de programação Python.

float + float

```
>>> a = 0.1
>>> b = 0.2
>>> c = a + b
>>> c
0.30000000000000004
```

```
int * float >>> e = 2 * b
>>> e
0.4
```



Ponto flutuante

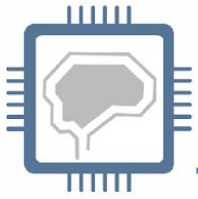
Os números de ponto flutuante ou ponto decimal são chamados de float no contexto da linguagem de programação Python.

float + float

```
>>> a = 0.1
>>> b = 0.2
>>> c = a + b
>>> c
0.30000000000000004
```

int * float

```
>>> e = 2 * b
>>> e
0.4
```



Ponto flutuante

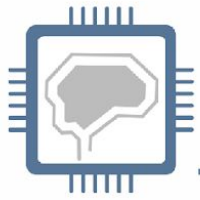
Se números inteiros e de ponto flutuante forem utilizados em uma mesma operação, o resultado será um tipo float.

```
>>> a = 1
>>> b = 2.0
→ >>> r2 = a + b
>>> r2
3.0
```

```
>>> r3 = 2 * 3.0
>>> r3
6.0
```

```
>>> r4 = 3.0 ** 2
>>> r4
9.0
```

```
>>> a = 4
>>> b = 2
>>> r1 = a/b
>>> r1
2.0
```



Ponto flutuante

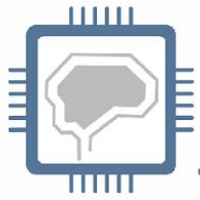
Se números inteiros e de ponto flutuante forem utilizados em uma mesma operação, o resultado será um tipo float.

```
>>> a = 1
>>> b = 2.0
>>> r2 = a + b
>>> r2
3.0
```

```
➔ >>> r3 = 2 * 3.0
>>> r3
6.0
```

```
>>> r4 = 3.0 ** 2
>>> r4
9.0
```

```
>>> a = 4
>>> b = 2
>>> r1 = a/b
>>> r1
2.0
```



Ponto flutuante

Se números inteiros e de ponto flutuante forem utilizados em uma mesma operação, o resultado será um tipo float.

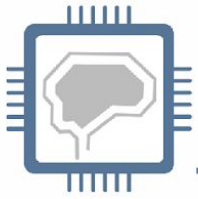
```
>>> a = 1
>>> b = 2.0
>>> r2 = a + b
>>> r2
3.0
```

```
>>> r3 = 2 * 3.0
>>> r3
6.0
```

→

```
>>> r4 = 3.0 ** 2
>>> r4
9.0
```

```
>>> a = 4
>>> b = 2
>>> r1 = a/b
>>> r1
2.0
```



Ponto flutuante

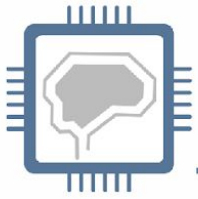
Se números inteiros e de ponto flutuante forem utilizados em uma mesma operação, o resultado será um tipo float.

```
>>> a = 1
>>> b = 2.0
>>> r2 = a + b
>>> r2
3.0
```

```
>>> r3 = 2 * 3.0
>>> r3
6.0
```

```
>>> r4 = 3.0 ** 2
>>> r4
9.0
```

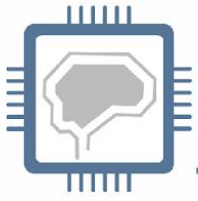
```
>>> a = 4
>>> b = 2
→ >>> r1 = a/b
>>> r1
2.0
```



Constantes

- O valor de uma **constante** deve permanecer o mesmo durante toda a execução de um programa.
- O Python **não** permite a definição de constantes.
- É uma prática comum em Python que os programadores usem **variáveis** com nome em letras **maiúsculas** para indicar que é uma constante.

```
>>> MAX_VALUE = 10
>>> MAX_VALUE
10
```

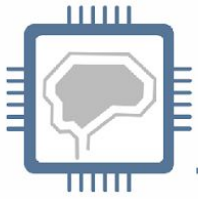



Texto

- Os strings são um conjunto de caracteres.
- Qualquer conjunto de caracteres entre aspas simples ou duplas é considerado um string em Python.

```
>>> mystring1 = "I have a son who is 6"  
>>> mystring1  
'I have a son who is 6'
```

```
>>> mystring2 = "ada lovelace"  
>>> mystring2  
'ada lovelace'
```



Texto

Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

```
>>> name = "ada lovelace"
```

→

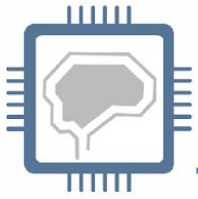
```
>>> name.title()  
'Ada Lovelace'
```

```
>>> name = name.upper()
```

```
>>> name  
'ADA LOVELACE'
```

```
>>> name.lower()  
'ada lovelace'
```

```
>>> name.upper()  
'ADA LOVELACE'
```



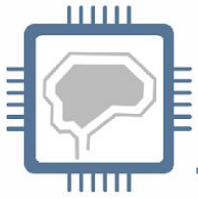
Texto

Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

```
>>> name = "ada lovelace"  
>>> name.title()  
'Ada Lovelace'
```

```
>>> name = name.upper()  
>>> name  
'ADA LOVELACE'  
>>> name.lower()  
'ada lovelace'
```

```
→ >>> name.upper()  
'ADA LOVELACE'
```



Texto

Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

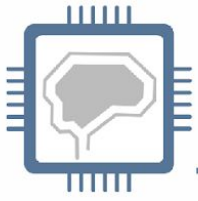
```
>>> name = "ada lovelace"  
>>> name.title()  
'Ada Lovelace'
```

```
>>> name = name.upper()  
>>> name  
'ADA LOVELACE'
```

→

```
>>> name.lower()  
'ada lovelace'
```

```
>>> name.upper()  
'ADA LOVELACE'
```

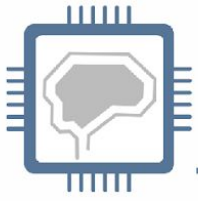


Texto

Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

f-strings

```
>>> first_name = "ada"
>>> last_name = "lovelace"
>>> full_name = f"{first_name} {last_name}"
>>> full_name
'ada lovelace'
```



Texto

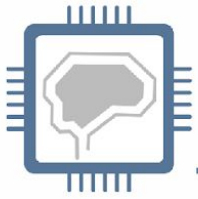
Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

Remove espaços
à direita

```
>>> favorite_language = ' python '  
>>> favorite_language.rstrip()  
' python '
```

```
>>> favorite_language.lstrip()  
'python '
```

```
>>> favorite_language.strip()  
'python '
```



Texto

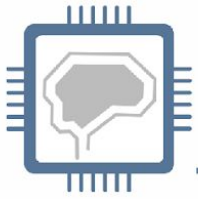
Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

```
>>> favorite_language = ' python '  
>>> favorite_language.rstrip()  
' python'
```

Remove espaços
à esquerda

```
>>> favorite_language.lstrip()  
'python '
```

```
>>> favorite_language.strip()  
'python'
```

Texto

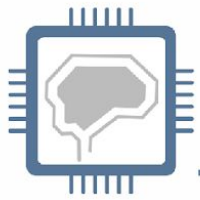
Python oferece grande flexibilidade para trabalhar com strings. Ilustraremos alguns usos de strings.

```
>>> favorite_language = ' python '  
>>> favorite_language.rstrip()  
' python '
```

```
>>> favorite_language.lstrip()  
'python '
```

Remove espaços à
direita e esquerda

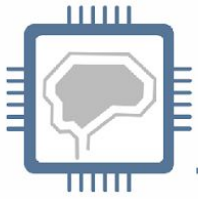
```
>>> favorite_language.strip()  
'python '
```

Operadores aritméticos

Já comentamos brevemente sobre as operações da Matemática que podemos realizar com o Python.

Operador	Descrição	Exemplo
()	Parênteses	$(x + y) * 2$
+	Adição	$x + y$
-	Subtração	$x - y$
*	Multiplicação	$x * y$
/	Divisão	x / y
%	Módulo (resto da divisão inteira)	$x \% y$
//	Quociente (da divisão inteira)	$x // y$
**	Potenciação (x elevado a y)	$x ** y$



Operadores aritméticos

Ilustramos aqui esses operadores:

```
>>> a=9
```

```
>>> b=4
```

```
>>> add = a + b
```

```
>>> add
```

```
13
```

```
>>> sub = a - b
```

```
>>> sub
```

```
5
```

```
>>> mul = a * b
```

```
>>> mul
```

```
36
```

```
>>> div1 = a / b
```

```
>>> div1
```

```
2.25
```

```
>>> div2 = a // b
```

```
>>> div2
```

```
2
```

```
>>> mod = a % b
```

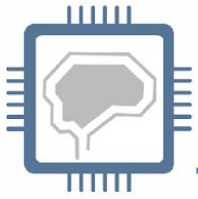
```
>>> mod
```

```
1
```

```
>>> p = a ** b
```

```
>>> p
```

```
6561
```



Operadores aritméticos

Ilustramos aqui esses operadores:

```
>>> a=9
```

```
>>> b=4
```

```
>>> add = a + b
```

```
>>> add
```

```
13
```

```
>>> sub = a - b
```

```
>>> sub
```

```
5
```

```
>>> mul = a * b
```

```
>>> mul
```

```
36
```

```
>>> div1 = a / b
```

```
>>> div1
```

```
2.25
```

```
>>> div2 = a // b
```

```
>>> div2
```

```
2
```

```
>>> mod = a % b
```

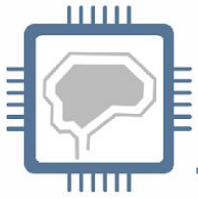
```
>>> mod
```

```
1
```

```
>>> p = a ** b
```

```
>>> p
```

```
6561
```



Operadores aritméticos

Ilustramos aqui esses operadores:

```
>>> a=9
>>> b=4
>>> add = a + b
>>> add
13
```

```
>>> sub = a - b
>>> sub
5
```

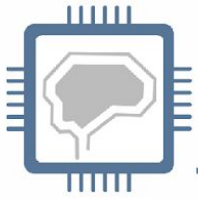
```
→ >>> mul = a * b
>>> mul
36
```

```
>>> div1 = a / b
>>> div1
2.25
```

```
>>> div2 = a // b
>>> div2
2
```

```
>>> mod = a % b
>>> mod
1
```

```
>>> p = a ** b
>>> p
6561
```



Operadores aritméticos

Ilustramos aqui esses operadores:

```
>>> a=9
>>> b=4
>>> add = a + b
>>> add
13
```

```
>>> sub = a - b
>>> sub
5
```

```
>>> mul = a * b
>>> mul
36
```

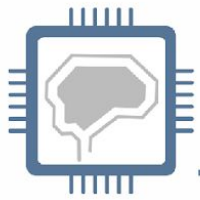
→

```
>>> div1 = a / b
>>> div1
2.25
```

```
>>> div2 = a // b
>>> div2
2
```

```
>>> mod = a % b
>>> mod
1
```

```
>>> p = a ** b
>>> p
6561
```



Operadores aritméticos

Ilustramos aqui esses operadores:

```
>>> a=9
>>> b=4
>>> add = a + b
>>> add
13

>>> sub = a - b
>>> sub
5

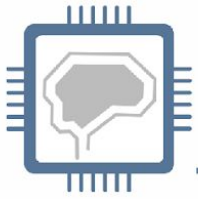
>>> mul = a * b
>>> mul
36

>>> div1 = a / b
>>> div1
2.25

>>> div2 = a // b
>>> div2
2

>>> mod = a % b
>>> mod
1

>>> p = a ** b
>>> p
6561
```

Operadores aritméticos

Ilustramos aqui esses operadores:

```
>>> a=9
>>> b=4
>>> add = a + b
>>> add
13

>>> sub = a - b
>>> sub
5

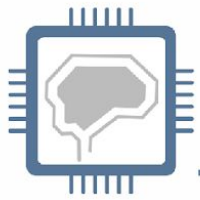
>>> mul = a * b
>>> mul
36

>>> div1 = a / b
>>> div1
2.25

>>> div2 = a // b
>>> div2
2

>>> p = a ** b
>>> p
6561

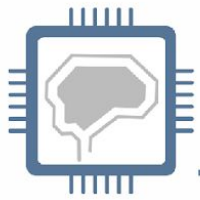
>>> mod = a % b
>>> mod
1
```

Operadores aritméticos

Ilustramos aqui esses operadores:

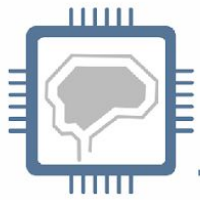
<pre>>>> a=9 >>> b=4 >>> add = a + b >>> add 13</pre>	<pre>>>> div1 = a / b >>> div1 2.25</pre>	 <pre>>>> p = a ** b >>> p 6561</pre>
<pre>>>> sub = a - b >>> sub 5</pre>	<pre>>>> div2 = a // b >>> div2 2</pre>	
<pre>>>> mul = a * b >>> mul 36</pre>	<pre>>>> mod = a % b >>> mod 1</pre>	



Operadores aritméticos

Atenção para a precedência dos operadores!

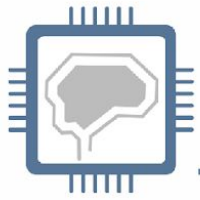
Operadores	Nomes dos operadores
()	Parênteses
**	Potenciação
*, /, //, %	Multiplicação, Divisão, Módulo
+, -	Adição, Subtração



Operadores relacionais

São aqueles utilizados para fazer a comparação de valores. O resultado dessa comparação pode ser **True** ou **False**.

Operador	Descrição	Exemplo
>	Maior que: True se o operando à esquerda é maior que o à direita	$x > y$
<	Menor que: True se o operando à esquerda é menor que o à direita	$x < y$
==	Igualdade: True se ambos operandos são iguais	$x == y$
!=	Diferente: True se os operandos são diferentes	$x != y$
>=	Maior ou igual: True se operando à esquerda é maior ou igual ao à direita	$x >= y$
<=	Menor ou igual: True se operando à esquerda é menor ou igual ao operando à direita	$x <= y$



Operadores relacionais

Vejamos alguns exemplos de operadores relacionais:

```
>>> a = 13
>>> b = 33
→ >>> a > b
False
```

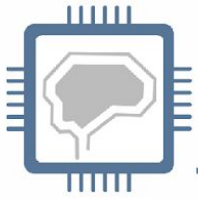
```
>>> a < b
True
```

```
>>> a != b
True
```

```
>>> a == b
False
```

```
>>> a >= b
False
```

```
>>> a <= b
True
>>>
```



Operadores relacionais

Vejamos alguns exemplos de operadores relacionais:

```
>>> a = 13
>>> b = 33
>>> a > b
False
```

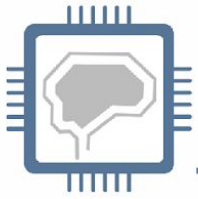
```
→ >>> a < b
True
```

```
>>> a != b
True
```

```
>>> a == b
False
```

```
>>> a >= b
False
```

```
>>> a <= b
True
>>>
```



Operadores relacionais

Vejamos alguns exemplos de operadores relacionais:

```
>>> a = 13
>>> b = 33
>>> a > b
False
```

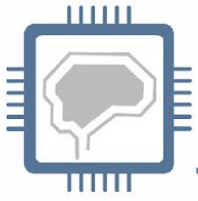
```
>>> a < b
True
```

```
→ >>> a != b
True
```

```
>>> a == b
False
```

```
>>> a >= b
False
```

```
>>> a <= b
True
>>>
```



Operadores relacionais

Vejamos alguns exemplos de operadores relacionais:

```
>>> a = 13
>>> b = 33
>>> a > b
False
```

```
>>> a < b
True
```

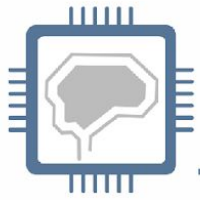
```
>>> a != b
True
```

→

```
>>> a == b
False
```

```
>>> a >= b
False
```

```
>>> a <= b
True
>>>
```

Operadores relacionais

Vejamos alguns exemplos de operadores relacionais:

```
>>> a = 13
>>> b = 33
>>> a > b
False
```

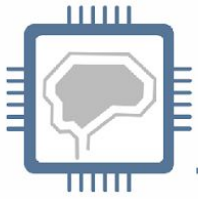
```
>>> a < b
True
```

```
>>> a != b
True
```

```
>>> a == b
False
```

```
→ >>> a >= b
False
```

```
>>> a <= b
True
>>>
```



Operadores relacionais

Vejamos alguns exemplos de operadores relacionais:

```
>>> a = 13
>>> b = 33
>>> a > b
False
```

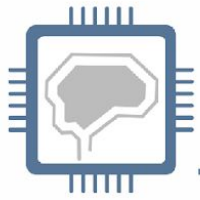
```
>>> a < b
True
```

```
>>> a != b
True
```

```
>>> a == b
False
```

```
>>> a >= b
False
```

```
→ >>> a <= b
True
>>>
```



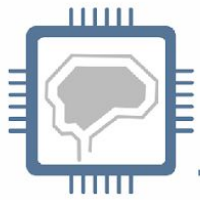
Comentários

São compostos por texto livre utilizado para explicar o programa que não é executado pelo interpretador Python.

À medida que os programas ficam maiores, é importante adicionar comentários.

Melhora a **legibilidade** do código!

```
>>> #say hello to everyone
... print("Hello Python people!")
Hello Python people!
...
```



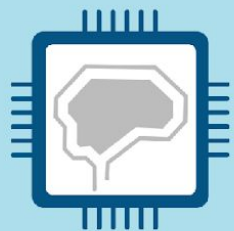
Indentação

Refere-se aos espaços que são colocados no início de uma linha de código.

Em várias linguagens de programação são úteis para a legibilidade do código.

Em **Python**, a indentação delimita um bloco de código e pode mudar o **escopo**, alterando a semântica do programa!

```
>>> if 5>2:
→ ...     print("Cinco é maior que 2")
    ...
    Cinco é maior que 2
```



Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira